

Module 13 — Edge Topics

Advanced techniques: pick and choose

What this module covers

- A **toolbox**, not a sequence — each topic stands alone, so pick what your system needs.
- Topics **1–11**: retrieval, grounding, safety and cost techniques — each with an exercise of the same number.
- Topics **12–14**: local ML on a laptop CPU — fine-tuning data, training, and running Hub models.
- Topics **15–18**: concepts to recognise — test-time compute, speculative decoding, GraphRAG, MoE.
- Evals (LLM-as-judge, retrieval metrics) have their own module — **Module 11**.



1. Hybrid Search

Keyword + semantic retrieval



What is hybrid search?

Hybrid search is a retrieval technique that combines keyword and semantic search. It is used in RAG when users mix exact terms (product codes, names) with paraphrased questions. You run **BM25** and **vector** search in parallel, merge the ranked lists (often with RRF), and pass the top passages to the LLM. The result is one ranked list that catches both literal matches and meaning-based matches.

Cormack, Clarke & Büttcher (2009). "Reciprocal Rank Fusion outperforms Condorcet and individual Rank Learning Methods." SIGIR.

BM25 scoring

$$\text{score}\left(D, Q\right) = \sum_{i=1}^{n} \text{IDF}\left(q_i\right) \cdot \frac{f\left(q_i, D\right) \cdot \left(k_1 + 1\right)}{f\left(q_i, D\right) + k_1 \cdot \left(1 - b + b \cdot \frac{|D|}{\text{avgdl}}\right)}$$

IDF weights rare terms higher. The fraction is term frequency with saturation ($k_1 \approx 1.5$) — repeated terms give diminishing returns. b (≈ 0.75) normalises by document length so long docs are not unfairly boosted.

Reciprocal Rank Fusion (RRF)

$$\text{score}\left(d\right) = \sum_{L \in \text{lists}} \frac{1}{k + \text{rank}\left(d, L\right)}$$

For each document, sum 1/(k + rank) across all ranked lists. Documents high in both lists score highest. k is typically 60. Score-agnostic — no normalisation across BM25 and cosine scores.

Hybrid search

- Combine **BM25** (keyword) and **vector** (semantic) search.
- Reciprocal Rank Fusion (RRF) merges ranked lists: `1/(k + rank)`.
- Score-agnostic — no normalisation needed across different scoring functions.
- Consistently outperforms either approach alone on real-world queries.



2. Re-ranking

Precision after broad retrieval



What is re-ranking?

Re-ranking is a second pass that re-sorts search results for accuracy. It is used when initial retrieval returns many candidates but the order is unreliable. A **re-ranker** (usually a small cross-encoder, not your chat LLM) scores each passage together with the question. The best passages move to the top, then the LLM generates its answer from those — so the model sees the right context first.

Nogueira & Cho (2019). "Passage Re-ranking with BERT." arXiv:1901.04085

↑↓ Re-ranking

- Initial retrieval is optimised for **recall**, not precision.
- Re-rank top-N with a **cross-encoder** or **LLM scorer**.
- Scores each (query, passage) pair together — catches nuance.
- Pipeline: retrieve 50 → re-rank 20 → generate from top 5.



3. HyDE

Hypothetical Document Embeddings



What is HyDE?

HyDE (Hypothetical Document Embeddings) is a retrieval trick for short or vague questions. It is used when the user's query looks nothing like the long documents in your index. The LLM writes a **hypothetical answer**, you embed that text, and search with that embedding instead of the raw question. Retrieval often improves because the fake answer sits closer to real passages in embedding space.

Gao et al. (2022). "Precise Zero-Shot Dense Retrieval without Relevance Labels." arXiv:2212.10496

HyDE — Hypothetical Document Embeddings

- Short queries land far from detailed documents in embedding space.
- Generate a **hypothetical answer** first, embed that instead.
- The hypothetical doc is structurally closer to real documents.
- Works best for knowledge-intensive domains with specialised vocab.



4. Agentic RAG

Retrieval as a tool



What is agentic RAG?

Agentic RAG is RAG where the LLM decides if and when to search, instead of always retrieving up front. It is used for mixed questions (some need documents, some do not) and multi-step research. You expose search as a **tool**; the agent calls it, reads results, and may search again. The answer is built only from retrieved evidence when needed — no wasted context on simple queries.

Jiang et al. (2023). "Active Retrieval Augmented Generation." arXiv:2305.06983

Agentic RAG

- Standard RAG: fixed retrieval step for every query.
- Agentic RAG: the LLM **decides** whether, when, and what to retrieve.
- Retrieval is a **tool** the agent calls on demand.
- Handles multi-hop questions and skips retrieval when unnecessary.



5. Citation Verification

Grounding answers in evidence



What is citation verification?

Citation verification is a post-generation check that grounds answers in sources. It is used when wrong or unsupported claims are unacceptable (legal, medical, finance). You **extract** claims from the answer, test each against retrieved passages, then flag or drop unsupported ones. The user gets an answer tied to evidence — or a refusal when the corpus does not support the claim.

Min et al. (2023). "FActScore: Fine-grained Atomic Evaluation of Factual Precision." arXiv:2305.14251

Citation verification

- LLMs generate plausible claims not backed by sources.
- **Extract** individual claims → **check** each against passages.
- Flag or remove **unsupported** claims before returning to user.
- Essential for high-stakes domains (legal, medical, financial).



6. Web Search Backend

When the answer is not in your corpus



What is a web search backend?

A **web search backend** adds live internet search as a fallback retrieval source. It is used when users ask about things your index does not cover — current events, other products, general knowledge. You search internally first; if the best score is below a threshold, you query a web search API and normalise the results (title, snippet, URL) into the same passage format, so generation does not care where they came from.

Nakano et al. (2021). "WebGPT: Browser-assisted question-answering with human feedback." arXiv:2112.09332

Web search backend

- Internal search first — your curated corpus stays the **authority**.
- **Fall back** to web search when the top internal score is below a threshold.
- Normalise web hits to the same `{text, source}` shape as internal passages.
- Trade-off: web results are unvetted, and add latency and an external dependency.



7. Text-to-SQL

Natural language over structured data



What is text-to-SQL?

Text-to-SQL turns natural language into database queries. It is used when answers live in tables (sales, logs, inventory), not in documents. You give the LLM the **schema**; it writes a SELECT, you run it safely (read-only), then return rows or a plain-language summary. The user asks in English and gets data-backed answers without writing SQL.

Yu et al. (2018). "Spider: A Large-Scale Human-Labeled Dataset for Complex Text-to-SQL." EMNLP.

Text-to-SQL

- Not all data lives in documents — much sits in **databases**.
- Inject the schema into the prompt; the LLM generates SELECT queries.
- **Safety**: read-only connections, forbidden keyword validation.
- Summarise results in natural language for better UX.



8. Advanced Guardrails

Defence in depth for production

What are advanced guardrails?

Guardrails are safety checks on what goes into and out of your LLM. They are used in production to block abuse, redact secrets, and enforce output shape. You chain filters on **input** (toxic content, PII) and **output** (PII again, schema validation). Unsafe or malformed content is blocked or cleaned before the user sees it.

Inan et al. (2023). "Llama Guard: LLM-based Input-Output Safeguard for Human-AI Conversations." arXiv:2312.06674

Advanced guardrails

- **Content filtering**: block toxic, harmful, or off-topic I/O.
- **PII detection**: redact emails, phone numbers, SSNs.
- **Schema validation**: Pydantic models enforce output structure.
- Defence in depth: chain multiple layers; no single guardrail is perfect.



9. Semantic Caching

Cache by meaning, not exact text



What is semantic caching?

Semantic caching stores past LLM answers and reuses them for similar questions. It is used to cut cost and latency when many users ask the same thing in different words. You embed the new query, find a close match in the cache (by cosine similarity), and return the stored answer if similarity is high enough. Otherwise you call the LLM and add the new pair to the cache.

Bang et al. (2023). "GPTCache: An Open-Source Semantic Cache for LLM Applications." arXiv:2308.15179

Cosine similarity

$$\cos\left(a, b\right) = \frac{a \cdot b}{|a| \ |b|}$$

Embed the incoming query, compare against cached query embeddings. If similarity exceeds a threshold (typically 0.95), return the cached response. Too low and you serve wrong answers; too high and the cache rarely hits.

Semantic caching

- Traditional caching needs exact key match — misses paraphrases.
- Semantic cache: embed query → find similar cached queries → return response.
- Similarity threshold (0.95) controls hit/miss sensitivity.
- Can serve 30-60% of queries from cache in repetitive workloads.



10. Multimodal RAG

Images, diagrams, and text together

What is multimodal RAG?

Multimodal RAG retrieves from images and diagrams as well as text. It is used when manuals, scans, or product photos matter as much as written docs. At index time, a vision model **describes** each image in text; you embed those descriptions like normal chunks. At query time, search returns text and image-derived passages so the LLM can answer questions about visuals.

Radford et al. (2021). "Learning Transferable Visual Models From Natural Language Supervision (CLIP)." ICML.

Multimodal RAG

- Knowledge bases contain **images**, diagrams, charts alongside text.
- Vision model generates text descriptions → embed → store in vector DB.
- Search returns both text chunks and image descriptions.
- Generate answers referencing both text and visual sources.



11. Contextual Chunking

How you split documents matters



What is contextual chunking?

Contextual chunking is how you split documents before indexing. It is used because bad splits break retrieval — sentences cut in half, topics split across chunks. You choose a strategy (fixed windows with overlap, parent-child sizes, or splits at topic boundaries), embed each chunk, and store metadata to link children to parents. Search hits precise chunks; generation can pull in the wider parent for context.

Lewis et al. (2020). "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks." NeurIPS.

Contextual chunking

- How you chunk has **massive** impact on retrieval quality.
- **Parent-child**: small chunks for search, large parents for context.
- **Overlapping windows**: redundancy at chunk boundaries.
- **Semantic chunking**: split at natural topic boundaries.



12. DPO + LoRA

Preference alignment + efficient training



What is DPO?

DPO (Direct Preference Optimization) is training that teaches a model which answers humans prefer. It is used for alignment — tone, safety, style — when prompting alone is not enough. You collect pairs of good vs bad responses and train with one preference loss (no separate reward model). Usually you use **LoRA** so only small adapter weights change. The model learns to favour the chosen style on similar prompts.

Rafailov et al. (2023). "Direct Preference Optimization: Your Language Model is Secretly a Reward Model." NeurIPS 2023.

LoRA — Low-Rank Adaptation

$$W^{'} = W + BA$$

Instead of updating all weights W, LoRA adds small trainable matrices B and A. W′ = W + BA. This is how DPO and other fine-tunes run efficiently — gigabytes of GPU memory, not terabytes.

Hu et al. (2021). "LoRA: Low-Rank Adaptation of Large Language Models." ICLR 2022.

DPO + LoRA

- **RLHF**: policy + reward + reference models, PPO optimisation loop.
- **DPO**: one loss on (chosen, rejected) preference pairs.
- **LoRA**: train only B and A adapters — same alignment, far less memory.
- Standard pipeline for open-source preference fine-tunes (Mixtral, Llama).
- Exercise `12-fine-tuning-data`: prepare the chat JSONL a fine-tune needs.



13. Train a Model Locally

CPU only — no GPU required



What is local training?

Local training means updating a small neural network on your own machine instead of calling a cloud API. It is used when you have labelled examples and want a cheap, repeatable classifier (intent, urgency, department). You prepare text + label pairs, fine-tune a base model like DistilBERT with Hugging Face **Trainer**, save weights to disk, then call **predict** on new text. You end up with a model file you own — no API key at inference time.

Wolf et al. (2020). "Transformers: State-of-the-Art Natural Language Processing." EMNLP System Demonstrations.

Local training workflow

- Install: `pip install -e ".[local-ml]"` (~2–4 GB RAM on CPU).
- Prepare labelled JSON: `{text, label}` pairs (demo: urgent/routine; exercise: department).
- Tokenise → **Trainer** with `max_steps=30` → save to `models/`.
- First run downloads **distilbert-base-uncased** (~250MB).
- Demo: `python demo/13_train_local.py` · Exercise: `exercises/13-local-training/`



Demo — Local training

python module-13-edge-topics/demo/13_train_local.py



14. Run a Hugging Face Model

Download once, run on CPU



What is running a Hub model?

The **Hugging Face Hub** hosts thousands of pre-trained models you download and run locally. It is used when you need a capability (sentiment, classification, translation) without training from scratch. You load a model id (e.g. DistilBERT fine-tuned on reviews), pass text through a **pipeline** or tokenizer + model, and read scores or labels. The model stays on disk in a cache — later runs start instantly.

Lhoest et al. (2021). "Datasets: A Community Library for Natural Language Processing." NeurIPS.

Hugging Face run workflow

- Install: `pip install -e ".[local-ml]"`.
- Demo: **`pipeline("sentiment-analysis")`** on ship logs — fastest path.
- Exercise: **AutoTokenizer** + **AutoModel** + softmax — see what the pipeline hides.
- Model: `distilbert-base-uncased-finetuned-sst-2-english` (~250MB download).
- Demo: `python demo/14_huggingface_run.py` · Exercise: `exercises/14-huggingface-run/`

Small models to try on CPU

- `distilbert-base-uncased-finetuned-sst-2-english` — sentiment (66M, ~250MB).
- `cardiffnlp/twitter-roberta-base-sentiment-latest` — 3-class sentiment (125M, ~500MB).
- `sentence-transformers/all-MiniLM-L6-v2` — sentence embeddings (22M, ~90MB).
- `facebook/bart-large-mnli` — zero-shot classification (407M, ~1.6GB).
- `Helsinki-NLP/opus-mt-en-fr` — English→French translation (74M, ~300MB).
- `google/flan-t5-small` — instruction-tuned text generation (77M, ~300MB).
- Rule of thumb: <500M params, "base"/"small"/"distil"/"mini" in the name → fine on CPU.



Demo — Hugging Face run

python module-13-edge-topics/demo/14_huggingface_run.py



15. Test-Time Compute Scaling

Think longer, not bigger

What is test-time compute scaling?

Normally an LLM answers in **one shot**: your question goes in, tokens come out. **Test-time compute scaling** means the model uses **more work at answer time** — extra tokens, steps, or attempts before you see the final reply. It is used for hard problems (math, logic, planning) where a quick first guess is often wrong. The system might write hidden reasoning, try several approaches, or check its own draft — like giving the model more exam time, not a bigger brain. You pay more latency and cost per question; you often get a better answer.

Snell et al. (2024). "Scaling LLM Test-Time Compute Optimally." arXiv:2408.03314

Test-time compute scaling

- **Train-time**: make the model bigger (more parameters, more GPU weeks).
- **Test-time**: same model, but **more inference steps** per user question.
- Examples: chain-of-thought, self-check, multiple samples + pick best (o1, DeepSeek-R1).
- Trade-off: answers can improve a lot; each query costs more time and money.



16. Speculative Decoding

Draft fast, verify once



What is speculative decoding?

Speculative decoding speeds up generation without changing the output. A small, fast **draft** model proposes several tokens; the large model checks them all in one forward pass and keeps the ones it agrees with. It is used in production serving where latency matters. Accepted tokens arrive in bulk — typically **2–4× faster** — and the final text is distributed exactly as if the large model had written it alone.

Leviathan, Kalman & Matias (2023). "Fast Inference from Transformers via Speculative Decoding." ICML.

Speculative decoding

- Draft model guesses **k tokens**; target model verifies them in **one pass**.
- Keep the agreed prefix, resample at the first mismatch, repeat.
- Same output distribution as the large model — only faster.
- Trade-off: memory for a second model; more complex serving stack.



17. GraphRAG

Knowledge graphs meet retrieval

What is GraphRAG?

GraphRAG is retrieval built on a knowledge graph, not just vector chunks. It is used for corpus-wide questions ("what are the main themes across these reports?"). You extract entities and links, group them into communities, and pre-summarise each cluster. A query can hit local chunks *or* those cluster summaries. You get both precise facts and big-picture answers.

Edge et al. (2024). "From Local to Global: A Graph RAG Approach to Query-Focused Summarization." arXiv:2404.16130

GraphRAG

- Extract entities + relationships → build a **knowledge graph**.
- **Community detection** + pre-summarise each cluster.
- Global queries answered at the cluster level, not chunk level.
- Combines local vector precision with global understanding.



18. Mixture of Experts

Massive parameters, selective compute

What is a Mixture of Experts model?

A **Mixture of Experts (MoE)** model has many specialist sub-networks but only runs a few per token. It is used to build very capable models without paying the full cost of every parameter on every token. A **router** picks top-K experts for each token; the rest stay idle. You get large-model quality with smaller active compute — e.g. Mixtral activates ~13B of ~47B parameters per token.

Jiang et al. (2024). "Mixtral of Experts." arXiv:2401.04088

Mixture of Experts

- Each block has **N expert** feed-forward layers instead of one.
- A learned **router** picks top-K experts per token.
- Mixtral 8×7B: ~47B total params, ~13B active per token.
- GPT-4 is widely believed to use MoE for scale and cost.



Putting it all together

Choosing techniques for your system

When to use what

Always use

- Hybrid search
- Re-ranking
- Evaluation (Module 11)
- Guardrails
- Good chunking strategy

Use when needed

- HyDE (specialised domains)
- GraphRAG (global / thematic queries)
- DPO + LoRA (preference alignment)
- Test-time compute (hard reasoning)
- Speculative decoding (latency)
- MoE awareness (model selection)
- Web search (out-of-corpus questions)
- Text-to-SQL (structured data)
- Semantic caching (high traffic)
- Multimodal (images in corpus)

Exercises — 14 hands-on topics

01–05: Hybrid search, re-ranking, HyDE, agentic RAG, citation verification

06–07: Web search backend, text-to-SQL

08–11: Advanced guardrails, semantic cache, multimodal RAG, contextual chunking

12–14: Fine-tuning data prep, **local training (CPU)**, **Hugging Face run**

Exercise numbers match topic numbers. Topics 15–18 are concepts only.



Capstone projects

extras/ — choose one, build it production-ready

Pick your capstone

AI Recipe Finder

Photo your fridge, get recipe suggestions.

RAG + hybrid search + multimodal vision.

Allergen guardrails + semantic caching.

AI Movie Night

Describe your mood, get movie picks.

RAG + reranking + text-to-SQL for data questions.

Charts and structured output.

AI Travel Planner

Describe your dream trip, get a day-by-day itinerary.

Agentic RAG + tool calling + web fallback.

Budget tracking + structured itineraries.

AI Personal Assistant

Friendly companion with calendar, notes, reminders.

Chat + tool calling + MCP + RAG over notes.

Session memory + cost controls.

How the capstones work

- Each capstone in `**extras/**` has a **frontend/** (SvelteKit — provided), a **starter/** (your code), and a **solution/**.
- You build the **FastAPI backend** in `starter/` — the frontend connects automatically.
- Combine **Module 12** patterns (tracing, retries, cost tracking) with **Module 13** techniques (hybrid search, caching, guardrails).
- Run with `uvicorn starter.app:app --reload --port 8000` and `cd frontend && pnpm dev`.



Module 13 — Complete

Next: pick a capstone in extras/